

Thực hành Dijkstra

```
1  #include <stdio.h>
2  #define INF 999999
3  #define MAXN 100
4
5  typedef struct {
6      int L[MAXN][MAXN];
7      int n, m;
8  }Graph;
9
10 void initGraph(Graph *pG, int n) {
11     int i, j;
12     pG->n = n;
13     pG->m = 0;
14     for (i = 1; i <= n; i++)
15         for (j = 1; j <= n; j++)
16             pG->L[i][j] = INF;
17 }
18
19 void addEdge(Graph *pG, int u, int v, int w) {
20     pG->L[u][v] = w;
21     pG->m ++;
22 }
23
24 //-----//
48 //-----//
49 #define NO_EDGE -1
50 int mark[MAXN];
51 int pi[MAXN];
52 int p[MAXN];
53 void Dijkstra(Graph* pG, int s) {
54     int i, j, e;
55     for (i = 1; i <= pG->n; i++) {
56         pi[i] = INF;
57         mark[i] = 0;
58     }
59     pi[s] = 0;
60     p[s] = -1;
61     for (e = 1; e < pG->n; e++) {
62         int min_pi = INF;
63         for (j = 1; j <= pG->n; j++)
64             if (mark[j] == 0 && pi[j] < min_pi) {
65                 min_pi = pi[j];
66                 i = j;
67             }
68         mark[i] = 1;
69         for (j = 1; j <= pG->n; j++)
70             if (pG->L[i][j] != NO_EDGE && mark[j] == 0) {
71                 if (pi[i] + pG->L[i][j] < pi[j]) {
72                     pi[j] = pi[i] + pG->L[i][j];
73                     p[j] = i;
74                 }
75             }
76     }
77 }
78
79 int main () {
80     // doc do thi co huong
81     freopen("D:/DoThi.txt", "r", stdin);
82     int n, m;
83     scanf("%d %d", &n, &m);
84     Graph G;
85     initGraph(&G, n);
86     int i;
87     for (i = 0; i < m; i++) {
88         int u, v, w;
89         scanf("%d %d %d", &u, &v, &w);
90         addEdge(&G, u, v, w);
91     }
92     Dijkstra(&G, 1);
93     for (i = 1; i <= G.n; i++)
94         printf("pi[%d] = %d, p[%d] = %d\n", i, pi[i], i, p[i]);
95 }
```

Giải thuật FLOYD

```
1  #include <stdio.h>
2  #define oo 999999
3  #define NO_EDGE -1
4  #define MAXN 100
5
6  typedef struct {
7      int n;
8      int W[MAXN][MAXN];
9  } Graph;
10
11 void init_graph(Graph* G, int n) {
12     G->n = n;
13     int i, j;
14     for (i = 1; i <= n; i++) {
15         for (j = 1; j <= n; j++) {
16             G->W[i][j] = NO_EDGE;
17         }
18     }
19 }
20
21 void add_edge(Graph* G, int u, int v, int w) {
22     G->W[u][v] = w;
23 }
24
25 int pi[MAXN][MAXN];
26 int next[MAXN][MAXN];
27
28 void FloydWarshall(Graph *pG) {
29     int u, v, k;
30     for (u = 1; u <= pG->n; u++)
31         for (v = 1; v <= pG->n; v++) {
32             pi[u][v] = oo;
33             next[u][v] = -1;
34         }
35     for (u = 1; u <= pG->n; u++)
36         pi[u][u] = 0;
37     for (u = 1; u <= pG->n; u++)
38         for (v = 1; v <= pG->n; v++)
39             if (pG->W[u][v] != NO_EDGE) {
40                 pi[u][v] = pG->W[u][v];
41                 next[u][v] = v;
42             }
43     for (k = 1; k <= pG->n; k++)
44         for (u = 1; u <= pG->n; u++)
45             for (v = 1; v <= pG->n; v++)
46                 if (pi[u][k] + pi[k][v] < pi[u][v]) {
47                     pi[u][v] = pi[u][k] + pi[k][v];
48                     next[u][v] = next[u][k];
49                 }
50 }
51
52 int main() {
53     Graph G, T;
54     int n, m, u, v, w, e;
55     freopen("D:/Bellman(KhongchuTrinh).txt", "r", stdin);
56     scanf("%d%d", &n, &m);
57     init_graph(&G, n);
58     for (e = 1; e <= m; e++) {
59         scanf("%d%d%d", &u, &v, &w);
60         add_edge(&G, u, v, w);
61     }
62     FloydWarshall(&G);
63     for (u = 1; u <= n; u++)
64         for (e = u+1; e <= n; e++)
65             if (pi[u][e] != oo)
66                 printf("%d -> %d : %d\n", u, e, pi[u][e]);
67     return 0;
68 }
```

Giải thuật Bellman

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define oo 999999
5  #define MAX_N 1000
6  #define MAX_M 1000
7
8  typedef struct {
9      int u, v, w;
10 } Edge;
11
12 typedef struct {
13     int n, m;
14     Edge edges[MAX_M];
15 } Graph;
16
17 int pi[MAX_N];
18 int p[MAX_N];
19
20 void BellmanFord(Graph *pG, int s, int t) {
21     int u, v, w, k, i;
22     for (u = 1; u <= pG->n; u++) {
23         pi[u] = oo;
24         p[u] = -1;
25     }
26     pi[s] = 0;
27
28     for (k = 1; k <= pG->n - 1; k++) {
29         for (i = 0; i < pG->m; i++) {
30             u = pG->edges[i].u;
31             v = pG->edges[i].v;
32             w = pG->edges[i].w;
33             if (pi[u] + w < pi[v]) {
34                 pi[v] = pi[u] + w;
35                 p[v] = u;
36             }
37         }
38     }
39     for (i = 0; i < pG->m; i++) {
40         u = pG->edges[i].u;
41         v = pG->edges[i].v;
42         w = pG->edges[i].w;
43         if (pi[u] + w < pi[v]) {
44             printf("Do thi chua chu trinh am!\n");
45             return;
46         }
47     }
48     printf("Duong ngan nhat tu %d den %d la %d\n", s, t, pi[t]);
49 }
50
51 int main() {
52     freopen("D:/Belman.txt", "r", stdin);
53     freopen("D:/Bellman(KhongchuTrinh).txt", "r", stdin);
54     Graph G;
55     int s, t, i;
56     scanf("%d %d", &G.n, &G.m);
57     for (i = 0; i < G.m; i++) {
58         scanf("%d %d %d", &G.edges[i].u, &G.edges[i].v, &G.edges[i].w);
59     }
60     //scanf("%d %d", &s, &t);
61     s = 1;
62     t = G.n;
63     BellmanFord(&G, s, t);
64     if (pi[t] == oo) {
65         printf("Khong co duong tu %d den %d\n", s, t);
66     } else {
67         printf("Duong ngan nhat tu %d den %d:\n", s, t);
68         int path[MAX_N];
69         int k = 0;
70         int current = G.n;
71         while (current != -1) {
72             path[k] = current; k++;
73             current = p[current];
74         }
75         for (i = k-1; i >= 0; i--)
76             printf("%d ", path[i]);
77     }
78
79     return 0;
80 }
```